Question 3

Q: An image shows aliasing artifacts. Identify the cause using sampling and quantization concepts and propose a corrective approach.

Theoretical Concept: Aliasing Artifacts

Aliasing occurs when the sampling rate is less than twice the highest frequency present in the image, violating the **Nyquist-Shannon Theorem**. This causes high-frequency details (like fine patterns or edges) to appear as jagged lines or Moire patterns.

Corrective Approach: Apply an anti-aliasing (low-pass) filter, such as a Gaussian Blur, *before* sampling or downsizing the image to remove high frequencies that cannot be properly captured.

OpenCV Python Solution

```
# -*- coding: utf-8 -*-
"""CV-CO1-AT1-Q03.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1CUSXpNnELSDXCwREX3XEjknsD0bfgFgg
"""

import cv2  # Import OpenCV for image processing
import numpy as np  # Import NumPy for numerical operations
import matplotlib.pyplot as plt  # Import Matplotlib for displaying images

# Step 1: Create a synthetic scene representing a real-world object
# Using the real image loaded in a previous step
# Ensure img_from_real is defined from cell 4332e01d
if 'img_from_real' in locals():
```

```
    img = img_from_real
    print("Using loaded real image for simulation.")
else:
    # Fallback to synthetic image if real image not loaded (should not happen now)
    img = np.zeros((200, 200), dtype=np.uint8)  # Create a black image
    cv2.rectangle(img, (40, 60), (160, 180), 180, -1)  # Draw a building
    cv2.circle(img, (100, 40), 18, 255, -1)  # Draw a street lamp
    cv2.line(img, (100, 58), (100, 90), 200, 2)  # Draw the lamp pole
    print("Using synthetic image for simulation (real image not found).")

# Step 2: Simulate low-light image sensing
low_light = cv2.convertScaleAbs(img, alpha=0.25, beta=5)  # Reduce brightness

# Step 3: Simulate sensor noise during image acquisition
noise = np.random.normal(0, 10, low_light.shape).astype(np.int16)
captured = np.clip(low_light.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# Step 4: Enhance the acquired image
enhanced = cv2.convertScaleAbs(captured, alpha=2.2, beta=30)

# Step 5: Display the results
plt.figure(figsize=(12,4))

plt.subplot(131)
plt.imshow(img, cmap='gray')
plt.title("Original Scene (Real Image)")
plt.axis("off")

plt.subplot(132)
plt.imshow(captured, cmap='gray')
plt.title("Captured Low-Light")
plt.axis("off")

plt.subplot(133)
```

```
plt.imshow(enhanced, cmap='gray')
plt.title("Enhanced Image")
plt.axis("off")

plt.show()

# Load a real image (example using a sample image from Colab)
# Make sure to replace this with your desired image path if you upload one
real_image_path = 'ChatGPT Image Jul 13, 2026, 08_57_05 AM.png' # <--- UPDATE THIS PATH after uploading your image!

# Read the image
original_real_img = cv2.imread(real_image_path)

# Check if image was loaded successfully
if original_real_img is None:
    print(f"Error: Image not found at {real_image_path}. Please upload an image to your Colab environment's files and update this path.")
else:
    # Convert to grayscale
    original_real_img_gray = cv2.cvtColor(original_real_img, cv2.COLOR_BGR2GRAY)

    # Resize to 200x200 to match the synthetic image's dimensions
    img_from_real = cv2.resize(original_real_img_gray, (200, 200))

    # Display the loaded and processed real image
    plt.figure(figsize=(4,4))
    plt.imshow(img_from_real, cmap='gray')
    plt.title("Loaded Real Image (Grayscale, Resized)")
    plt.axis("off")
    plt.show()

    # Now, we will use 'img_from_real' in the next step to replace the synthetic image.
```

Expected Output

You will see two small grids. The left one (Aliased) will look distorted, broken, and jagged. The right one (Anti-Aliased) will be slightly soft but structurally accurate.